

BC Parks Asset Image Classification Installation Guide

Capstone Project Handoff

June 18, 2026

Contents

Purpose	2
What This Project Produces	2
Repository Setup	2
Python Environment	2
Credentials	3
DINOv3 Model Weights	3
Data Inputs	3
Optional CityWide Download	4
Prepare Clean Images	4
Run The Final Pipeline	4
Run A Fast Smoke Check	5
Predict On A Folder Of New Images	5
Optional Cloud VLM Pipeline	5
Important Output Locations	5
Cleaning Generated Files	6
Troubleshooting	6
Missing DINOv3 Weights	6
Missing Cleaned Image Marker	6
Missing CityWide Credentials	6
Environment Problems	7

Purpose

This guide explains how to install and run the BC Parks asset image classification pipeline from this repository.

It assumes that the workstation already has the basic development tools installed, such as Visual Studio Code, Git, a terminal, and Miniforge or Conda. For a brand-new Windows workstation, use the separate BC Parks software setup guide first.

What This Project Produces

The final pipeline produces partner-facing CSV files with predicted asset attributes and confidence scores:

```
results/final/bcparks_asset_attribute_predictions_long.csv
results/final/bcparks_asset_attribute_predictions_wide.csv
```

The main model path uses local image files, DINOv3 image embeddings, and lightweight supervised classifiers. Optional CityWide and cloud VLM steps are available when credentials are provided.

Repository Setup

Clone the repository:

```
git clone https://github.com/sgauth01/parks-asset-img-class.git
cd parks-asset-img-class
```

If the repository is already cloned, update it:

```
git pull
```

Python Environment

Create the Conda environment:

```
conda env create -f environment.yml
conda activate bcparks_capstone
```

If the environment already exists, update it instead:

```
conda env update -f environment.yml --prune
conda activate bcparks_capstone
```

Check that Python can import the core packages:

```
make env-check
```

If `make` is not available on Windows, install it in the Conda environment:

```
conda install -n bcparks_capstone -c conda-forge make
```

Credentials

Copy the example environment file:

```
cp .env.example .env
```

Open `.env` in a text editor and fill only the credentials needed for the commands you plan to run. The `.env` file is ignored by Git and should not be committed.

Required only for CityWide downloads:

```
CITYWIDE_API_KEY=  
CITYWIDE_DB=  
CITYWIDE_USER=  
CITYWIDE_API_URL=https://v4.citywidesolutions.com/v4_server/external/v1
```

Required only for optional cloud VLM predictions:

```
GEMINI_API_KEY=  
GOOGLE_API_KEY=  
OPENAI_API_KEY=  
ANTHROPIC_API_KEY=  
XAI_API_KEY=  
GITHUB_TOKEN=
```

DINOv3 Model Weights

The DINOv3 model weights are not committed to Git because they are large and access-controlled.

Request access from Meta and download:

```
dinov3_vitb16_pretrain_lvd1689m-73cec8be.pth
```

Place the file here:

```
models/downloaded_model/dinov3_vitb16_pretrain_lvd1689m-73cec8be.pth
```

The default Makefile variable points to that exact location:

```
DINO_WEIGHTS=models/downloaded_model/dinov3_vitb16_pretrain_lvd1689m-73cec8be.pth
```

Data Inputs

The final pipeline expects the processed data files in:

```
data/processed/master_dataset.csv  
data/processed/train/  
data/processed/attribute_applicability.csv
```

The repository tracks the processed metadata and training CSVs. Large raw images and generated feature files are intentionally not committed.

Check the required processed inputs:

```
make data-check
```

Optional CityWide Download

Use this only if BC Parks wants to refresh raw CityWide metadata and images.

Check CityWide credentials:

```
make citywide-check
```

Download metadata and attributes only:

```
make download-citywide-metadata
```

Download metadata, attributes, attached-file metadata, and images:

```
make download-citywide-images
```

Run a small download smoke test:

```
make download-citywide-sample
```

CityWide outputs are written under:

```
data/raw/citywide/
```

For the full CityWide API flow, including how linked attributes are downloaded, see:

```
docs/citywide_api_runbook.md
```

Prepare Clean Images

Before running the final model pipeline on a fresh checkout, build the cleaned image set:

```
make pii
```

This step:

1. Screens raw images for possible PII.
2. Blurs flagged images.
3. Builds the cleaned local image set under:

```
data/processed/images_clean/
```

It also writes a completion marker:

```
data/processed/images_clean/.upload_set_complete
```

The final pipeline checks for this marker before running model inference.

Run The Final Pipeline

Run the full final DINOv3 pipeline:

```
make all
```

This is equivalent to:

```
make final-dinov3
```

The pipeline runs tests, checks inputs, confirms the cleaned image set exists, builds or reuses DINOv3 features, trains final classifiers, and exports the partner-facing CSV files.

Run A Fast Smoke Check

Use this before a heavier model run:

```
make smoke
```

This runs environment checks, data checks, unit tests, and baseline logic.

Predict On A Folder Of New Images

For a small built-in demo:

```
make demo
```

For a custom folder:

```
make predict-new-images NEW_IMAGE_FOLDER=data/raw/citywide/images
```

If the folder is flat and all images are one asset type, provide the asset type:

```
make predict-new-images NEW_IMAGE_FOLDER=/path/to/images NEW_IMAGE_ASSET_TYPE="Trail Bridge"
```

Optional Cloud VLM Pipeline

The VLM pipeline is optional and requires provider credentials.

Run a small VLM smoke test:

```
make vlm-smoke VLM_PROVIDER=gemini VLM_MODEL=gemini-3-flash-preview
```

Run final DINOv3 plus an optional VLM branch:

```
make final-with-vlm VLM_PROVIDER=gemini VLM_MODEL=gemini-3-flash-preview
```

Important Output Locations

Final partner-facing CSVs:

```
results/final/bcparks_asset_attribute_predictions_long.csv
results/final/bcparks_asset_attribute_predictions_wide.csv
```

DINOv3 feature files:

```
data/features/
```

Baseline and evaluation outputs:

```
results/baseline_results/
results/dinov3_results/
```

PII review outputs:

```
results/predictions/pii_screen.csv
data/pii_review/
data/processed/images_clean/
```

Cleaning Generated Files

Remove final prediction CSVs:

```
make clean-final
```

Remove generated DINOv3 features for the current model:

```
make clean-dinov3
```

Remove both final predictions and generated DINOv3 features:

```
make clean
```

Troubleshooting

Missing DINOv3 Weights

If the pipeline reports a missing DINOv3 file, confirm this file exists:

```
models/downloaded_model/dinov3_vitb16_pretrain_lvd1689m-73cec8be.pth
```

Missing Cleaned Image Marker

If the pipeline reports:

```
Missing data/processed/images_clean/.upload_set_complete
```

run:

```
make pii
```

Then rerun:

```
make all
```

Missing CityWide Credentials

If CityWide commands fail with missing credentials, confirm `.env` contains:

```
CITYWIDE_API_KEY
CITYWIDE_DB
CITYWIDE_USER
```

Then rerun:

```
make citywide-check
```

Environment Problems

If imports fail, update the environment:

```
conda env update -f environment.yml --prune
conda activate bcparks_capstone
```

Then rerun:

```
make smoke
```

Security And Data Notes

- Do not commit `.env`.
- Do not commit raw CityWide images.
- Do not commit DINOv3 model weights.
- Do not commit generated feature files.
- Use the cleaned image set for model runs and cloud VLM runs.
- Review PII outputs before sharing image-derived artifacts externally.